

# 8.27 模拟面试

## 如何实现视频播放的跳转？

跳转操作可以直接使用 FFMPEG 的跳转函数 `av_seek_frame` 来实现，函数原型如下：

```
1  int av_seek_frame(AVFormatContext *s, int stream_index, int64_t
    timestamp, int flags);
```

| 参数名                  | 类型                                         | 说明                                                   |
|----------------------|--------------------------------------------|------------------------------------------------------|
| s                    | AVFormatContext*                           | 打开的媒体文件上下文（由 <code>avformat_open_input</code> 返回）    |
| stream_index         | int                                        | 指定流的索引，若为 <code>-1</code> ，则表示自动选择一个默认流（通常是主视频流）     |
| timestamp            | int64_t                                    | 目标时间戳，单位是 <code>AVStream.time_base</code> （即对应流的时间基） |
| flags                | int                                        | 控制定位行为的标志位，常用的有以下几种：                                 |
| 标志位宏名                | 含义                                         |                                                      |
| AVSEEK_FLAG_BACKWARD | 定位到小于或等于 <code>timestamp</code> 的关键帧       |                                                      |
| AVSEEK_FLAG_ANY      | 可以定位到任意帧（不一定是关键帧）                          |                                                      |
| AVSEEK_FLAG_FRAME    | <code>timestamp</code> 以“帧数”为单位，而不是以时间戳为单位 |                                                      |
| AVSEEK_FLAG_BYTE     | <code>timestamp</code> 以字节为单位（慎用，效率        |                                                      |

| 参数名 | 类型 | 说明 |
|-----|----|----|
|     | 低) |    |

- 不能直接简单的将当前 帧开始放入音、视频队列中进行处理，因为音视频队列中还有之前的数据，这些数据还可以播放几秒，因此首先需要清理 队列中这些数据
- 直接这样处理，会导致跳转时的花屏现象

原因：解码器中还保留了上一帧的信息，跳转后，会对当前帧产生影响，因此还需要清理解码器中的数据；

实现：往队列中放入一个特殊的 packet `#define FLUSH_DATA "FLUSH"`，当解码线程取到这个 packet 的时候，就执行清除解码器的数据

- 关键帧 (I帧) 问题：由于视频解码需要关键帧才能解码出有效信息，因此跳转时，是跳转到关键帧；

例如 10——15s之间，关键帧是第 10s 和第15s,如果要跳转到 第13s秒开始播放，实际上是跳转到第 10s，将第10——12s的数据扔掉，播放第13s的数据；

在run函数的循环中，处理跳转标志时，`av_seek_frame` 只能跳转到关键帧，（这里就会存在误差）同时向音、视频缓冲队列放入清除的包 `FLUSH`；

为了解决 `av_seek_frame` 的跳转误差，因此需要在音视频各自的解码线程中解决关键帧但未到目的帧的帧舍弃问题；

**注意：**并且在处理清除包 `FLUSH` 时一定要将音频时钟 `video_clock` 置0，否则 `video_clock`一直记录的时跳转之前的时间00:05:00，在经过同步算法 `synchronize_video` 后 pts 也与 `video_clock` 保持了一致是 00:05:00.xx，音频时钟此时才 00:01:30.xx，差距巨大 → 音视频同步算法会误以为要丢 3 分多钟的帧；画面直接黑屏或出现长时间“等待音频追赶”。

关键帧：不依赖其他帧即可独立解码的视频帧；也就是说I帧本身就是完整的画面，无需再参考前后帧再解析；

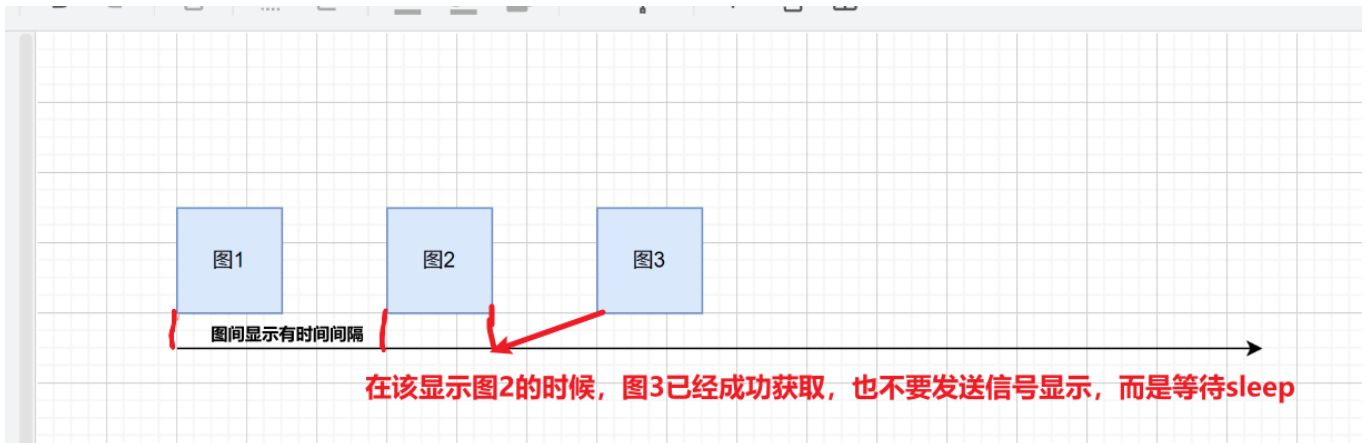
关键：

- 解码的起点：播放器必须先拿到一个I帧才能开始正确解码后序P/B帧

- 跳转：会找最近的I帧开始解码
- 错误恢复：网络包丢失or损坏后，下一个I帧可以重置画面，防止错误累积；

## 如何实现音视频同步???

图片显示是有时间间隔的；图片显示是按照时间轴来，因此即使图片提前加载出来，没有到达时间轴的时间也不要显示，那么就sleep；



由于图片之间显示有固定的时间间隔，因此可以推导什么时候开始显示对应的图片，只有当时间轴上的时间大于等于这个计算出的时间时（用时间戳表示），才能显示对应的图片；

### 简易同步思路

在播放视频的时候，我们发现视频较快，会出现过快，与时间不同步，解决办法，每次解析的时候，都记录时间(显示时间戳)，  
然后判断当前视频的时间，比较真实时间和显示时间，(该方法下，视频会超前)，  
一旦显示时间更大，那么就等一下(`msleep(10)`)，从而解决视频与时间不同步的问题。

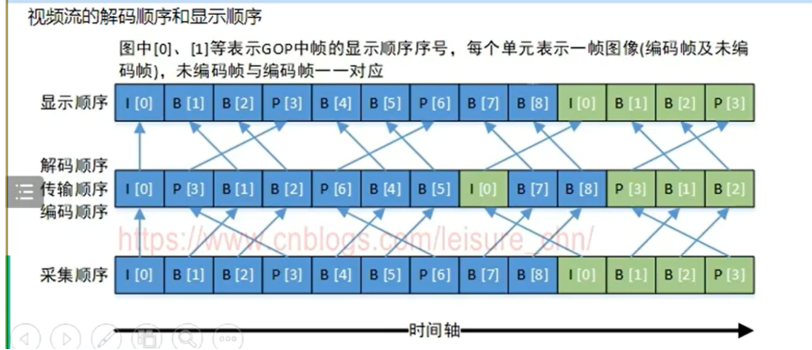
# 简易同步

## I帧/P帧/B帧

**I帧**: I帧(Intra-coded picture, 帧内编码帧, 常称为关键帧)包含一幅完整的图像信息, 属于帧内编码图像, 不含运动矢量, 在解码时不需要参考其他帧图像。因此在I帧图像处可以切换频道, 而不会导致图像丢失或无法解码。

**P帧**: P帧(Predictive-coded picture, 预测编码图像帧, 常称为差异帧)是帧间编码帧, 利用之前的I帧或P帧进行预测编码。

**B帧**: B帧(Bi-directionally predicted picture, 双向预测编码图像帧, 常称为双向差异帧)是帧间编码帧, 利用之前和(或)之后的I帧或P帧进行双向预测编码。B帧不可以作为参考帧。B帧具有更高的压缩率, 但需要更多的缓冲时间以及更高的CPU占用率, 因此B帧适合本地存储以及视频点播, 而不适用对实时性要求较高的直播系统。



PTS: 显示时间戳。

DTS: 解码时间戳。

同步采用 PTS

因此同步只需要让 音频和视频的播放都与主机时间轴(主时钟)保持一致;

即在循环前获取一次系统时间 `start_time=av_gettime()`, `pts=0`;//当前视频时间;

读取完视频帧后, 再获取一遍系统时间 `av_gettime()`, 当前视频理论的显示时间  
`pts= pFrame->pts = pFrame->best_effort_timestamp`

实际实现音视频同步是以 音频时钟作为 主时钟, 每次解码音频的时候, 都计算音频时钟, 每一次解码视频前, 都获取一下当前视频对应显示时间 `video_clock(pts)`, 并且有一个同步线程保证 视频的理论显示时间 `pts` 和 实际应该显示的视频实际 `video_clock` 保持一致, 如果发现视频显示时间大于音频的时间, 那么就延时 (`SDL_Delay(5)`), 通过这种方式, 来让视音频同步. 让视频追赶音频;

问题1: 为什么修改为音频时钟作为主时钟?

系统时钟只显示“当前几点”, 但不会驱动 声卡和显卡;

声卡晶振只能以固定频率震荡, 其本身不能丢样本或者变速度, 影响因素只有 制造、温度、老化等; 但是系统时间会因为电源、矫正等问题, 随时改变此时的时间, 频率变化不是稳定的;

问题2: 为什么不是视频作为主时钟?

因为视频的容易发生丢帧和重复帧，变速也没有明显感觉，显示器的刷新频率也是变化的；而声卡**固定采样率时钟**，丢/插样本会立刻听到爆音/杂音，因此音频难以被调整，让容易被修改的视频追赶音频最简单；

## PTS DTS 分别是什么？

PTS (**Presentation Time Stamp**) 是显示时间戳；

DTS (**Decoding Time Stamp**) 是解码时间戳；（同步我们使用的是PTS）

为什么需要两个时间戳？

（I帧 P帧 B帧）需要“先解码后显示”，视频里存在**双向参考帧（B 帧）**，即需要参考前后的I帧和P帧

播放器按 **DTS** 把数据送进解码器，再按 **PTS** 把解码后的帧送进渲染队列/声卡。